interview-thalamus-sweep

Documentation Index

Contents

- **Complexity map** — `docs/architecture-complexity-map.md` (short reader guide for the Thalamus / Sweep / Fish split).
- **Architecture specs** — `docs/specs/architecture/` (14 chapters: ontology, design stance, layout, thalamus, sweep, SSA primary build, transpositions, three swarms, shared foundation, design choices, running locally, consoles, references, package onboarding).
- **Module specs** — `docs/specs/` (per-package LaTeX specs with acceptance criteria and traceability tables; see `specs/README.md`).
- **Tasks & plans** — `docs/tasks/`.
- **Superpowers** — `docs/superpowers/` (ignored by default; reference only).

Architecture reading map

The fastest way to read this repository is to keep three loops separate: **Thalamus** creates confidence-gated knowledge graph findings, **Sweep** audits and reviews suggestions through a human-in-the-loop resolution path, and **Fish** are isolated counterfactual `sim_run` workers inside Sweep. A fish owns its agents, turns, and memory; cross-fish meaning appears only when the aggregator clusters terminal summaries under a quorum rule. This distinction prevents the common misread that "swarm", "finding", and "suggestion" are interchangeable words.

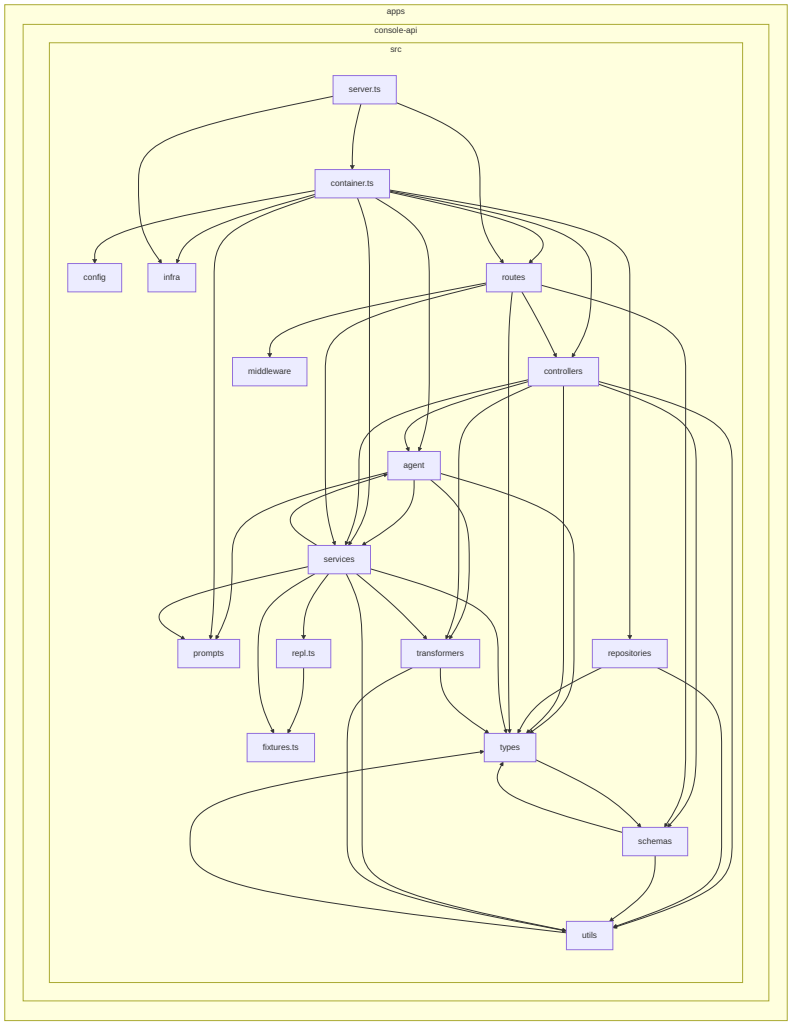
For the compact version, start with `architecture-complexity-map.md`, then read SPEC-ARCH-01, SPEC-ARCH-04, SPEC-ARCH-05, and SPEC-ARCH-08.

Per-package dependency graphs

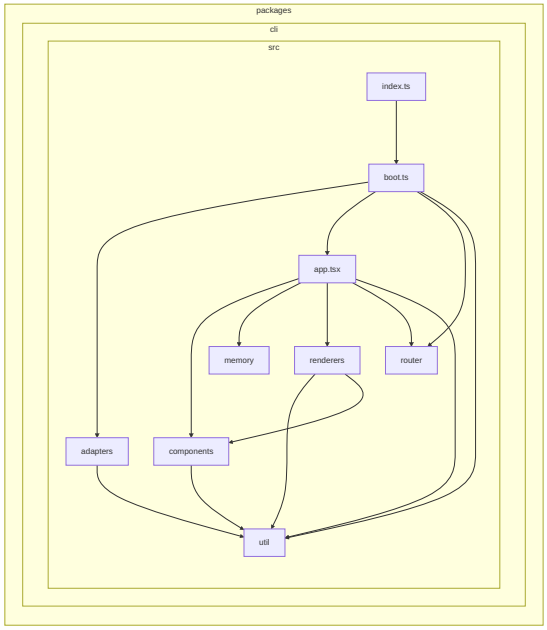
Each graph below is generated by `dependency-cruiser` on the target package's `src/` tree, collapsed one directory level under `src/`. Sources live in `docs/figures/dependency-graph/` (`.mmd` + `.pdf` pair per package).

apps/console

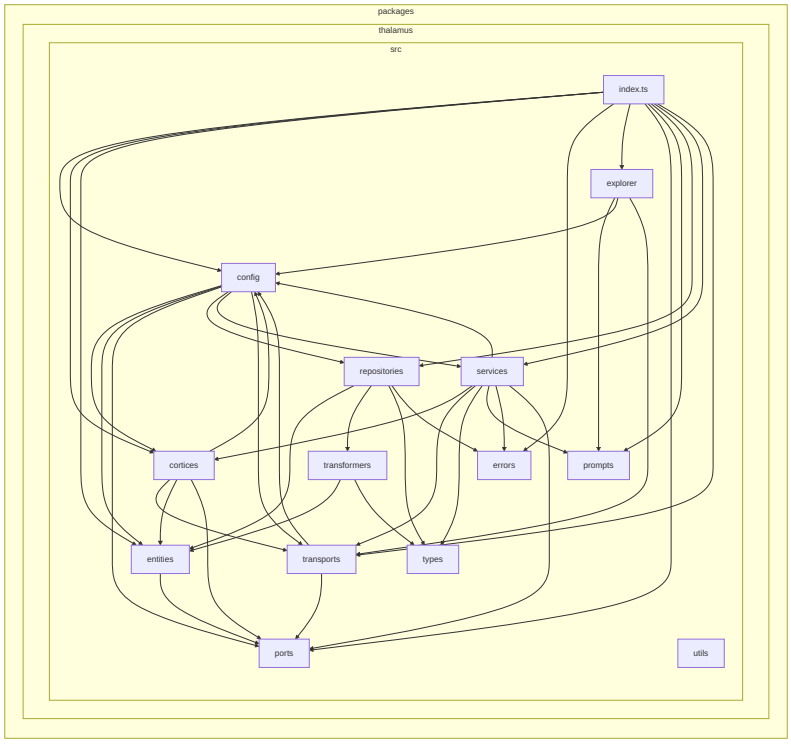
apps/console-api



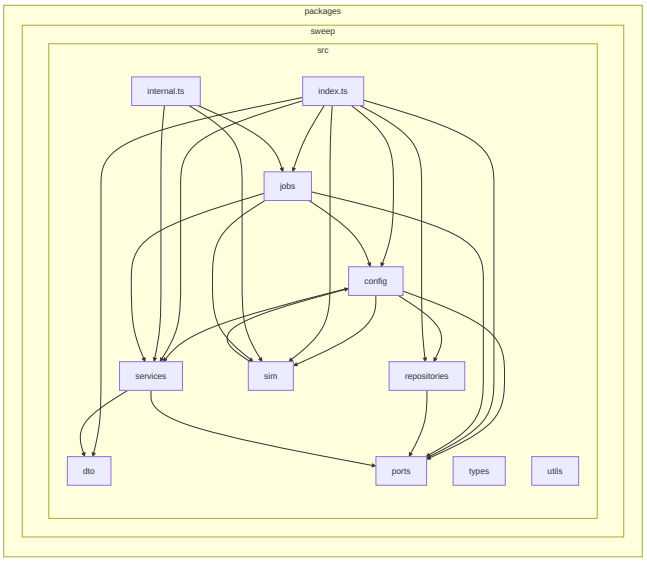
packages/cli



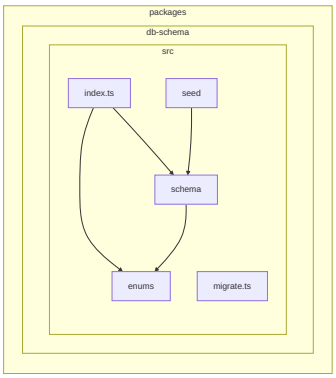
packages/thalamus



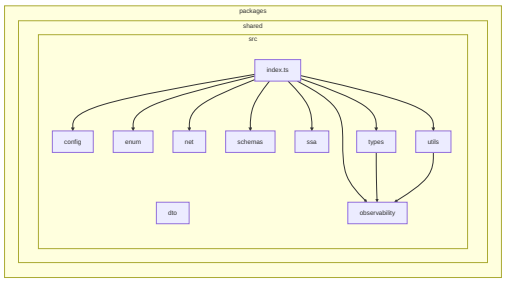
packages/sweep



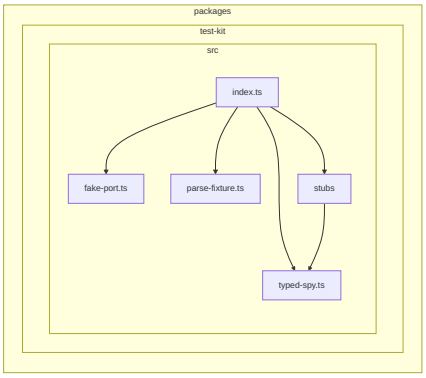
packages/db-schema



packages/shared

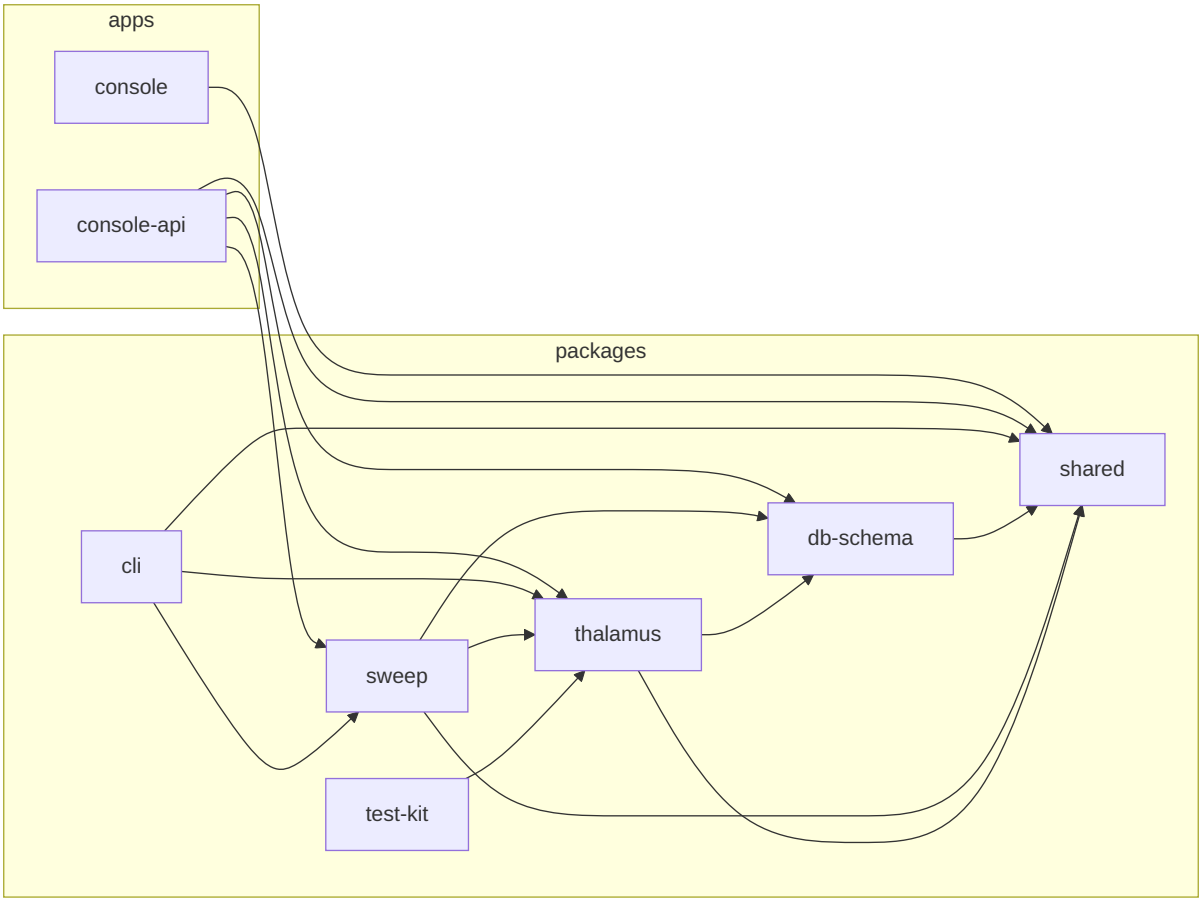


packages/test-kit



Repository dependency graph — general view

High-level view of the monorepo's first-party dependencies (apps → packages and packages → packages), generated from `.dependency-cruiser.js` with `node_modules` and Node core collapsed out.



Source: docs/dependency-graph.mmd (Mermaid) PDF: docs/dependency-graph.pdf

Regenerate. From the repository root, one-shot for all 8 app/package source trees + the general view:

```
for p in apps/console apps/console-api packages/cli packages/db-schema \
    packages/shared packages/sweep packages/test-kit packages/thalamus; do \
      name=${p##*/}; \
```

```
      pnpm exec depcruise -config .dependency-cruiser.js \
        -output-type mermaid -include-only "^$p/src/" \
        -collapse "^$p/src/[^/]+" $p/src \
        > docs/figures/dependency-graph/$name.mmd; \
      pnpm exec mmdc -i docs/figures/dependency-graph/$name.mmd \
        -o docs/figures/dependency-graph/$name.pdf -b transparent; \
done
pnpm exec depcruise -config .dependency-cruiser.js -output-type mermaid \
     -include-only "^(apps|packages)/" \
     -collapse "^(apps|packages)/[^/]+" apps packages \
     > docs/dependency-graph.mmd
pnpm exec mmdc -i docs/dependency-graph.mmd -o docs/dependency-graph.pdf -b transparent
```

Architectural invariants

See `CLAUDE.md` (repo root) for the non-negotiable boundary rules. The operational contract is the `console-api` HTTP surface; remaining CLI in-process calls use public package APIs and are called out explicitly in the architecture specs until the CLI is fully HTTP-only. The dependency graph above is the structural witness for those package boundaries.